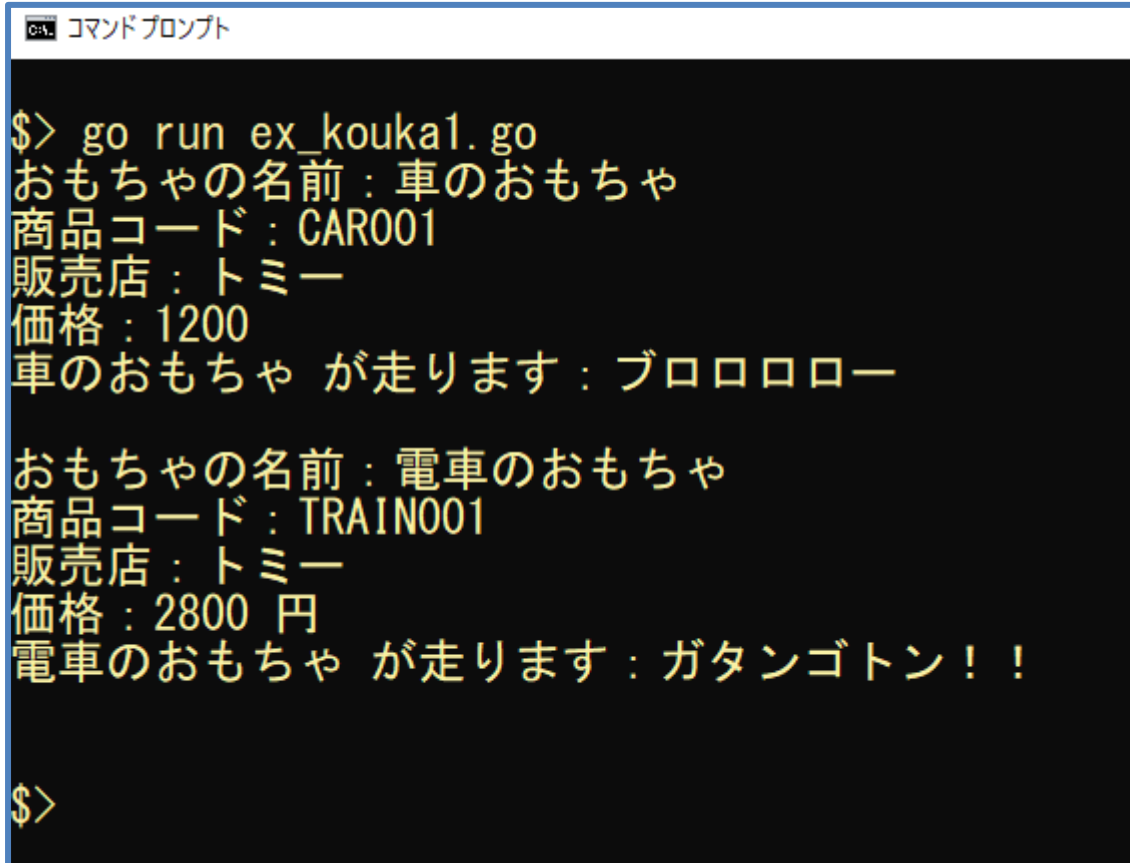


問題1

下記の実行結果となるように未完成プログラムの空欄部分を埋めてプログラムをプログラムを完成させてください。

◆構造体を使用します。

実行結果



```
cmd コマンドプロンプト

$> go run ex_kouka1.go
おもちゃの名前：車のおもちゃ
商品コード：CAR001
販売店：トミー
価格：1200
車のおもちゃ が走ります：ブロロロー

おもちゃの名前：電車のおもちゃ
商品コード：TRAIN001
販売店：トミー
価格：2800 円
電車のおもちゃ が走ります：ガタンゴトン！！

$>
```

未完成プログラム ファイル名:ex_kouka1.go

```
package main
```

```
import "fmt"
```

```
/**
```

```
構造体 CarToy の定義
```

```
フィールド
```

```
    name おもちゃの名前(型 string)  
    code おもちゃのコード(型 string)  
    outlet おもちゃの販売元(型 stirng)  
    price 定価(型 int)
```

```
*/
```

プログラム作成部分

```
/**
```

```
構造体 TrainToy の定義
```

```
フィールド
```

```
    name おもちゃの名前(型 string)  
    code おもちゃのコード(型 string)  
    outlet おもちゃの販売元(型 stirng)  
    price 定価(型 int)
```

```
*/
```

プログラム作成部分

```
/**
```

```
メソッド displayToy の定義
```

```
メソッド名:displayToy
```

```
レシーバ:CarToy 構造体のポインタ
```

```
引数:なし
```

```
処理:
```

```
    CarToy 構造体のフィールドを表示
```

```
戻り値:なし
```

```
*/
```

プログラム作成部分

```
/**
メソッド run の定義
メソッド名:run
レシーバ:CarToy 構造体のポインタ
引数:なし
処理:車のおもちゃの走る音を表示
戻り値:なし
*/
```

プログラム作成部分

```
/**
メソッド displayToy の定義
メソッド名:displayToy
レシーバ:TrainToy 構造体のポインタ
引数:なし
処理:
    TrainToy 構造体のフィールドを表示
戻り値:なし
*/
```

プログラム作成部分

```
/**
メソッド run の定義
メソッド名:run
レシーバ:TrainToy 構造体のポインタ
引数:なし
処理:電車のおもちゃの走る音を表示
戻り値:なし
*/
```

プログラム作成部分

```
func main() {
    /**
    CarToy 構造体を初期化
    変数名 carToy
    名前:車のおもちゃ
    商品コード:CAR001
    販売元:トミー
    価格:1200
    */
```

```
_____(1)_____  
  
/**  
TrainToy 構造体を初期化  
変数名 trainToy  
名前:電車のおもちゃ  
商品コード:TRAIN001  
販売元:トミー  
価格:2800  
*/  
  
_____(2)_____  
  
/**  
構造体 carToy の displayToy メソッドを呼び出して  
フィールドの情報を表示  
*/  
  
_____(3)_____  
// carToy の run メソッドを呼び出す  
  
_____(4)_____  
fmt.Println()  
/**  
構造体 trainToy の displayToy メソッドを呼び出して  
フィールドの情報を表示  
*/  
  
_____(5)_____  
// trainToy の run メソッドを呼び出す  
  
_____(6)_____  
fmt.Println()  
}
```

問題2

問題1のプログラムを修正します。構造体の埋め込みを利用して、実行結果が問題1と同じになるように、未完成プログラムの空欄部分を埋めてプログラムを完成させてください。

実行結果

```
$> go run ex_kouka2.go
おもちゃの名前：車のおもちゃ
商品コード：CAR001
販売店：トミー
価格：1200
車のおもちゃ が走ります：ブロロロー

おもちゃの名前：電車のおもちゃ
商品コード：TRAIN001
販売店：トミー
価格：2800 円
電車のおもちゃ が走ります：ガタンゴトン！！

$>
```

未完成プログラム ファイル名:ex_kouka2.go

```
package main
```

```
import "fmt"
```

```
/**
構造体 Toy の定義
フィールド
    name おもちゃの名前(型 string)
    code おもちゃのコード(型 string)
*/
```

プログラム作成部分

```
/**
構造体 CarToy の定義
フィールド
    Toy Toy 構造体の埋め込み
    outlet おもちゃの販売元(型 string)
    price 定価(型 int)
*/
```

プログラム作成部分

```
/**
構造体 TrainToy の定義
フィールド
    Toy Toy 構造体の埋め込み
    outlet おもちゃの販売元(型 string)
    price 定価(型 int)
*/
```

プログラム作成部分

```
/**
メソッド displayToy の定義
メソッド名:displayToy
レシーバ:Toy 構造体のポインタ
引数:なし
処理:
    Toy 構造体のフィールドを表示
戻り値:なし
```

```
*/
```

プログラム作成部分

```
/**
```

メソッド displayToy の定義

メソッド名:displayToy

レシーバ:CarToy 構造体のポインタ

引数:なし

処理:

CarToy 構造体のフィールドを表示

戻り値:なし

```
*/
```

プログラム作成部分

```
/**
```

メソッド run の定義

メソッド名:run

レシーバ:CarToy 構造体のポインタ

引数:なし

処理:車のおもちゃの走る音を表示

戻り値:なし

```
*/
```

プログラム作成部分

```
/**
```

メソッド displayToy の定義

メソッド名:displayToy

レシーバ:TrainToy 構造体のポインタ

引数:なし

処理:

TrainToy 構造体のフィールドを表示

戻り値:なし

```
*/
```

プログラム作成部分

```
/**
メソッド run の定義
メソッド名:run
レシーバ:TrainToy 構造体のポインタ
引数:なし
処理:電車のおもちゃの走る音を表示
戻り値:なし
*/
```

プログラム作成部分

```
func main() {
    /**
    CarToy 構造体を初期化
    変数名 carToy
    名前:車のおもちゃ
    商品コード:CAR001
    販売元:トミー
    価格:1200
    */
    _____(1)_____

    /**
    TrainToy 構造体を初期化
    変数名 trainToy
    名前:電車のおもちゃ
    商品コード:TRAIN001
    販売元:トミー
    価格:2800
    */
    _____(2)_____

    /**
    構造体 carToy の displayToy メソッドを呼び出して
    フィールドの情報を表示
    */
    _____(3)_____
    // carToy の run メソッドを呼び出す
    _____(4)_____
    fmt.Println()
    /**
    構造体 trainToy の displayToy メソッドを呼び出して
    フィールドの情報を表示
    */
    _____(5)_____
    // trainToy の run メソッドを呼び出す
    _____(6)_____
    fmt.Println()
}
```


問題3

問題2のプログラムを、インタフェースを利用することにより実装します。実行結果は問題2と同じとなります。

未完成プログラムの空欄部分を埋めてプログラムを完成させてください。

実行結果

```
$> go run ex_kouka3.go
おもちゃの名前：車のおもちゃ
商品コード：CAR001
販売店：トミー
価格：1200
車のおもちゃ が走ります：ブロロロー

おもちゃの名前：電車のおもちゃ
商品コード：TRAIN001
販売店：トミー
価格：2800 円
電車のおもちゃ が走ります：ガタンゴトン！！

$>
```

未完成プログラム ファイル名:ex_kouka3.go

```
package main
```

```
import "fmt"
```

```
/**  
ToyProc インタフェースの定義  
メソッド:  
    displayToy  
    run  
*/
```

プログラム作成部分

```
/**  
構造体 Toy の定義  
フィールド  
    name おもちゃの名前(型 string)  
    code おもちゃのコード(型 string)  
*/
```

プログラム作成部分

```
/**  
構造体 CarToy の定義  
フィールド  
    Toy Toy 構造体の埋め込み  
    outlet おもちゃの販売元(型 string)  
    price 定価(型 int)  
*/
```

プログラム作成部分

```
/**  
構造体 TrainToy の定義  
フィールド  
    Toy Toy 構造体の埋め込み  
    outlet おもちゃの販売元(型 string)  
    price 定価(型 int)  
*/
```

プログラム作成部分

```
/**
メソッド displayToy の定義
メソッド名:displayToy
レシーバ:Toy 構造体のポインタ
引数:なし
処理:
    Toy 構造体のフィールドを表示
戻り値:なし
*/
```

プログラム作成部分

```
/**
メソッド run の定義
メソッド名:run
レシーバ:Toy 構造体のポインタ
引数:なし
処理:おもちゃの走る音を表示
戻り値:なし
*/
```

プログラム作成部分

```
/**
メソッド displayToy の定義
メソッド名:displayToy
レシーバ:CarToy 構造体のポインタ
引数:なし
処理:
    CarToy 構造体のフィールドを表示
戻り値:なし
*/
```

プログラム作成部分

```
/**
メソッド run の定義
メソッド名:run
レシーバ:CarToy 構造体のポインタ
引数:なし
処理:車のおもちゃの走る音を表示
戻り値:なし
*/
```

プログラム作成部分

```
/**
メソッド displayToy の定義
メソッド名:displayToy
レシーバ:TrainToy 構造体のポインタ
引数:なし
処理:
    TrainToy 構造体のフィールドを表示
戻り値:なし
*/
```

プログラム作成部分

```
/**
メソッド run の定義
メソッド名:run
レシーバ:TrainToy 構造体のポインタ
引数:なし
処理:電車のおもちゃの走る音を表示
戻り値:なし
*/
```

プログラム作成部分

```
func main() {
    /**
    CarToy 構造体を初期化
    変数名 carToy
    名前:車のおもちゃ
    商品コード:CAR001
    販売元:トミー
    価格:1200
    */
}
```

(1)

```
/**  
TrainToy 構造体を初期化  
変数名 trainToy  
名前:電車のおもちゃ  
商品コード:TRAIN001  
販売元:トミー  
価格:2800  
*/
```

(2)

```
/**  
インタフェース ToyProc の宣言  
変数名:toyProc  
*/
```

(3)

```
// 変数 toyProc に carToy 構造体のアドレス(参照)を代入
```

(4)

```
// インタフェースを使用して、displayToy メソッドを呼び出す
```

(5)

```
// インタフェースを使用して、run メソッドを呼び出す
```

(6)

```
fmt.Println()
```

```
// 変数 toyProc に trainToy 構造体のアドレス(参照)を代入
```

(7)

```
// インタフェースを使用して、displayToy メソッドを呼び出す
```

(8)

```
// インタフェースを使用して、run メソッドを呼び出す
```

(9)

```
fmt.Println()
```

```
}
```